

RAPPORT D'AUDIT FORENSIQUE

AGENTFORGE

Audit Complet - 10 Domaines

52.2 / 100

Niveau de Maturite : BETA

AGENTFORGE

Rapport d'Audit Forensique

Cabinet d'Audit Compose d'Architectes Staff+,
Experts SaaS Enterprise, Ingenieurs DevSecOps,
SRE Google/AWS, Auditeurs OWASP

Score Global : 52.2 / 100

Niveau de Maturite : BETA

Date : 4 juin 2026 | Reference : AF-AUDIT-2026-001
Methode : Zero supposition - Preuves techniques uniquement

Table des Matieres

1. Resume Executif
2. Scores par Module
3. Audit 1 : Architecture Globale (62/100)
4. Audit 2 : API Exhaustif (58/100)
5. Audit 3 : Agents IA (87/100)
6. Audit 4 : Administration (49/100)
7. Audit 5 : Providers IA (72/100)
8. Audit 6 : Multi-Tenant (42/100)
9. Audit 7 : Securite Enterprise (55/100)
10. Audit 8 : Observabilite (35/100)
11. Audit 9 : Performance (30/100)
12. Audit 10 : Production Readiness (32/100)
13. Failles Critiques
14. Modules Non Fonctionnels
15. Fonctionnalites Operationnelles
16. Ameliorations Prioritaires
17. Niveau de Maturite

1. Resume Executif

Cabinet d'audit : Architectes logiciels Staff+, experts SaaS Enterprise, ingenieurs DevSecOps, experts IA generative, SRE Google/AWS, auditeurs OWASP et specialistes des plateformes multi-tenant. Methodologie appliquee : aucune affirmation acceptee sans preuve issue du code, des tests, des logs, des bases de donnees ou des executions reelles. Chaque constat est documente avec la reference du fichier source et la ligne de code correspondante.

AgentForge est une plateforme d'agents IA orchestres en architecture monorepo (pnpm/Turborepo) composant de 4 packages : @agentforge/api (Hono + Drizzle + PostgreSQL), @agentforge/web (Vite + React 19), @agentforge/shared (types, schemas, utils), @agentforge/sandbox (Docker). Le code source totalise environ 26 500 lignes reparties sur 159 fichiers. La plateforme presente un noyau d'orchestration IA remarquablement sophistique (MoA 3-round cascading, Reflection Agent avec LLM reel, RL-adaptive Router), mais souffre d'un probleme systemique majeur : de nombreux services enterprise sont implements mais jamais integres dans le chemin d'execution de l'application.

Le score global de 52.2/100 reflète cette realite : le coeur IA operationnel (MoA, Reflection, RL) contraste avec des modules enterprise entierement deconnectes (observabilite, queue, deploiement, multi-tenant). La plateforme est actuellement au niveau **BETA** : le produit central fonctionne, mais l'infrastructure enterprise necessite une integration massive avant toute mise en production.

Module	Score	Statut
Architecture Globale	62/100	PARTIEL
API Exhaustif	58/100	PARTIEL
Agents IA	87/100	OPERATIONNEL
Administration	49/100	PARTIEL
Providers IA	72/100	PARTIEL
Multi-Tenant	42/100	DEFAILLANT
Securite Enterprise	55/100	PARTIEL
Observabilite	35/100	DEFAILLANT
Performance	30/100	DEFAILLANT
Production Readiness	32/100	DEFAILLANT
GLOBAL (moyenne)	52.2/100	BETA

Tableau 1 : Synthèse des scores par module d'audit

3. Audit 1 : Architecture Globale

Score: 62/100

3.1 Architecture Reelle (basee sur le code)

L'architecture reelle d'AgentForge suit un modele en couches : Frontend React (7 pages) -> API Hono (6 groupes de routes, 82 endpoints) -> Core Engine (DAG + MoA + Reflection) -> PostgreSQL (Drizzle ORM) + Redis (ioredis) + LLM Providers (9 modeles). L'orchestration utilise un DAG avec tri topologique et execution parallele par niveau via Promise.allSettled. Le flux d'evenements transite par EventManager (singleton) vers les clients SSE. Le cache fonctionne en deux niveaux : L1 (Map memoire, TTL 5min) et L2 (Redis, TTL 1h, namespace agentforge:cache:*).

3.2 Services Morts Detectes

L'audit a identifie 6 services completement morts (exportes mais jamais appeles par aucune route ou service actif) et 3 middleware deconnectes. Ces services representent environ 2 000+ lignes de code mort qui augmentent la surface d'attaque et la dette technique sans apporter de valeur fonctionnelle. Le probleme systemique est l'integration gap : les services sont bien concus et implements, mais personne ne les a branche dans le chemin d'execution.

Service	Lignes	Raison du statut mort
SecretRotationService	158	initialize() et startPeriodicCheck() jamais appeles
AstTransformService	546	Exporte mais jamais importe par ReflectionAgent
JobQueue	530	Aucune route n'enfile de jobs ; execution synchrone uniquement
DistributedCache	414	L'ancien CacheManager est utilise a la place
RequestDeduplicator	366	Aucune route n'applique la deduplication
SandboxManager	519	Importe dans admin.ts mais jamais instancie
tenantMiddleware	238	Defini mais jamais applique a aucune route
requiredTenantMiddlewa re	-	Defini mais jamais applique a aucune route
optionalAuth	-	Defini mais jamais applique a aucune route

Tableau 2 : Services et middleware morts detectes

3.3 Donnees Multi-Tenant Non Persistees

Les services `TenantService`, `TenantQuotaService`, `TenantBillingService` et `RBACService` fonctionnent entierement en memoire via des objets `Map`. Les tables PostgreSQL (`tenants`, `tenantMembers`) existent dans `schema.ts` mais ne sont jamais ecrites par les services au runtime. Un redemarrage du serveur entraine la perte totale de toutes les donnees de tenancy, facturation et quotas. C'est un risque critique pour toute utilisation en production.

3.4 Pages Frontend Manquantes

Le frontend ne couvre que 35% de l'API disponible. Les 7 pages existantes (`Dashboard`, `Agents`, `AgentDetail`, `Execute`, `Projects`, `Login`, `Register`) ne representent que les fonctionnalites de base. Six domaines majeurs de l'API n'ont aucune representation frontend : MFA (5 endpoints), Tenants (14 endpoints), Deploiements (10 endpoints), Admin (24 endpoints), Feedback (2 endpoints), et Coûts/Predictions (3 endpoints). Le fichier `api.ts` ne contient aucune methode pour ces API.

4. Audit 2 : API Exhaustif

Score: **58/100**

Metrique	Valeur	Evaluation
Total endpoints	82	Couverture etendue
Endpoints avec authentication	70/82 (85.4%)	Correct mais lacunes
Endpoints avec validation Zod	24/82 (29.3%)	INSUFFISANT
Endpoints avec gestion erreurs	33/82 (40.2%)	INSUFFISANT
Endpoints avec rate limiting	2/82 (2.4%)	CRITIQUE
CORS wildcard sur SSE	Oui	Vulnerabilite
Endpoints 501 Not Implemented	3	Stubs admin

Tableau 3 : Metriques API globales

4.1 Vulnerabilites Critiques API

ID	Endpoint	Risque	Description
CRIT-1	POST /telemetry/alerts/:id/acknowledge	CRITIQUE	Mutation d'etat sans authentication - n'importe qui peut acquitter des alertes
CRIT-2	POST /admin/providers/:id/test	HAUT	Fuite du prefix (8 car.) et longueur des cles API
CRIT-3	GET /telemetry/status, GET /telemetry/alerts	HAUT	Endpoints sans auth exposant la configuration OTLP et les alertes
CRIT-4	POST /auth/login, POST /auth/register	HAUT	Aucun rate limiting - vulnérable au brute force
CRIT-5	POST /tenants/	HAUT	N'importe quel utilisateur authentifie peut creer des tenants
CRIT-6	GET /projects/:id	MOYEN	Pas de verification de propriete - IDOR
CRIT-7	Tous /deployments/*	MOYEN	Aucune verification de propriete sur les 10 endpoints

Tableau 4 : Vulnerabilites critiques de l'API

4.2 Couverture Validation par Groupe de Routes

La validation des entrees via Zod est critique pour la securite (OWASP A08). La couverture actuelle de 29.3% est insuffisante. Les groupes les plus defaillants

sont : Admin (4% - seul PATCH /users a une validation), Top-level (0% - endpoints de telemetrie sans validation), et Agents (29% - seuls feedback, predictions et execute sont valides). Les routes d'authentification sont a 80%, mais refresh et logout n'ont aucune validation et planteront sur du JSON invalide.

5. Audit 3 : Agents IA

Score: 87/100

Le module Agents IA est le point fort d'AgentForge avec un score de 87/100. L'orchestration MoA (Mixture of Agents) implemente un veritable cycle Propose (3-9 LLMs) -> Critique (2-3 LLMs) -> Refine (3 LLMs) avec fusion intelligente, deduplication Jaccard, et vote multicriteres (qualite 40%, cout 30%, vitesse 30%). Le Reflection Agent utilise de vrais appels LLM via Gemini 2.0 Flash avec 6 criteres ponderes. L'auto-fix en 3 niveaux (Pattern -> AST -> LLM) fonctionne reellement avec escalation. Le RL Router est authentiquement adaptatif avec epsilon-greedy (5% initial, decay 0.9995), moteur de recompense multi-facteur, et persistance en base de donnees. C'est le module le plus mature de la plateforme.

5.1 Verification des Cles IA

Fonctionnalite	Statut	Preuve Code
MoA 3-round cascading	OPERATIONNEL	CodeGenerator.ts lignes 129-303
9 modeles paralleles (DEV)	OPERATIONNEL	agents.ts lignes 107-109, providerCount: 9
Vote multicriteres	OPERATIONNEL	CodeGenerator.ts lignes 506-602
Fusion intelligente Jaccard	OPERATIONNEL	CodeGenerator.ts lignes 613-672
Reflection LLM reel	OPERATIONNEL	ReflectionAgent.ts ligne 91, Gemini 2.0 Flash
6 criteres ponderes	OPERATIONNEL	agents.ts lignes 29-36
Auto-fix 3 niveaux	OPERATIONNEL	SuperAgent.ts lignes 527-571
Epsilon-greedy RL	OPERATIONNEL	RLTrainingService.ts lignes 66-75
Persistence RL en DB	OPERATIONNEL	RLTrainingService.ts lignes 138-156
Apprentissage continu	PARTIEL	checkAndRetrain() jamais appele automatiquement

Tableau 5 : Verification des fonctionnalites IA

5.2 Lacunes Identifiees

Malgre le score eleve, deux lacunes significatives persistent. Premierement, `ContinuousLearningService.checkAndRetrain()` n'est jamais declenche automatiquement ; il faut un appel manuel via `POST /agents/learning/retrain`. Deuxiemement, l'estimation de confiance dans le MoA est purement heuristique (non verifiee par LLM), et la boucle de feedback est limitee a un seul cycle de raffinement supplementaire. Enfin, le consensus early-exit fonctionne mais repose uniquement sur des scores heuristiques sans validation LLM croisee.

6. Audit 4 : Administration

Score: **49/100**

Module Admin	Score	Lacunes Principales
User Management	9/15	Pas de recherche, pas de reset mot de passe, pas de gestion des roles
Agent Management	4/10	Read-only, PATCH retourne 501, pas d'activation/desactivation
Provider Management	2/10	PATCH retourne 501, test = verification cle uniquement, fuite prefix cle
Quota Management	7/10	Service complet mais pas d'endpoint admin dedie
Financial Management	8/10	Pas d'integration Stripe, paiement auto-complete = simule
Security Management	5/10	Pas de gestion MFA/session depuis admin
Infrastructure	6/10	Pas de gestion deploiement, alertes sans API acknowledge/resolve
Audit Logs	6/10	Filtres service existants mais non exposes via API admin
Frontend Admin	0/10	ZERO pages admin - API uniquement
Auth/Z Admin	2/5	Auth = tier entreprise uniquement, pas de RBAC

Tableau 6 : Scores detaillés du module Administration

Le module administration souffre de trois problemes fondamentaux. Premièrement, l'authentification admin verifie uniquement le tier (entreprise) et non le role RBAC. N'importe quel utilisateur avec le tier entreprise peut acceder a toutes les fonctions admin, y compris la suppression d'utilisateurs et la consultation des donnees financieres. Deuxièmement, trois endpoints admin critiques retournent 501 Not Implemented : PATCH /admin/agents, PATCH /admin/providers, et PATCH /admin/orchestration/config. Troisièmement, il n'existe aucune interface web admin ; toutes les operations necessitent des appels API directs (curl/Postman).

7. Audit 5 : Providers IA

Score: 72/100

Provider	Implementation	Appel HTTP Reel	Cle API Env
OpenAI (gpt-4o, gpt-o1)	callOpenAI()	Oui - fetch()	OPENAI_API_KEY
Anthropic (claude-3.7-sonnet)	callAnthropic()	Oui - fetch()	ANTHROPIC_API_KEY
Google (gemini-2.5-pro, gemini-2.0-flash)	callGoogle()	Oui - fetch()	GOOGLE_API_KEY
DeepSeek (deepseek-r1)	callOpenAICompatible()	Oui - fetch()	DEEPSEEK_API_KEY
Mistral (mistral-large)	callOpenAICompatible()	Oui - fetch()	MISTRAL_API_KEY
Together (llama-3.3, qwen-2.5)	callOpenAICompatible()	Oui - fetch()	TOGETHER_API_KEY / DASHSCOPE_API_KEY
OpenRouter	ABSENT	Non	N/A
Ollama (local)	ABSENT	Non	N/A

Tableau 7 : Statut des providers IA

7.1 Gestion des Cles API

La gestion des cles API presente des lacunes significatives. Les cles sont stockees uniquement en variables d'environnement (pas de vault, pas de chiffrement au repos). L'interface admin permet uniquement la lecture du statut configure/non configure, mais ne permet pas d'ajouter ou modifier des cles (PATCH /admin/providers retourne 501). Le test de connexion verifie seulement l'existence de la cle, sans effectuer d'appel LLM reel. De plus, le endpoint de test fuite les 8 premiers caracteres de la cle API et sa longueur totale, facilitant les attaques par brute force cibles. Le SecretRotationService est implemente mais jamais initialise au demarrage de l'application.

8. Audit 6 : Multi-Tenant

Score: **42/100**

Le module multi-tenant est le plus defaillant de la plateforme avec un score de 42/100. Le probleme fondamental est que les services de tenancy (TenantService, RBACService, TenantQuotaService, TenantBillingService) sont entierement bases sur des Maps en memoire. Les tables de base de donnees (tenants, tenantMembers) existent dans le schema Drizzle mais ne sont jamais lues ni ecrites par ces services. En consequence, toute donnee de tenancy est perdue au redemarrage du serveur.

8.1 Isolation Tenant

L'isolation entre tenants est pratiquement inexistante. Il n'y a pas de Row-Level Security PostgreSQL, pas de schemas separes par tenant, et aucun filtrage par tenantId dans les requetes des routes projects, agents ou deployments. Le tenantMiddleware et requiredTenantMiddleware sont implements mais jamais appliques a aucune route. Les quotas ne sont verifies que sur un seul endpoint (POST /:id/users), laissant les executions d'agents sans aucune limitation. La RBAC est partiellement enforcee dans les routes tenants mais totalement absente des routes projects, agents et deployments.

8.2 Facturation

La facturation est fonctionnelle en memoire avec generation de factures, calcul de depassement, enregistrement de paiements et changements de plan. Cependant, il n'y a aucune integration avec un prestataire de paiement reel (Stripe, etc.). Les paiements sont automatiquement marques comme "completed" sans verification. La taxe est figuree a 0% en dur. Toutes les donnees de facturation sont perdues au redemarrage.

9. Audit 7 : Securite Enterprise

Score: **55/100**

OWASP Top 10	Statut	Details
A01 - Broken Access Control	CRITIQU E	IDOR sur GET /projects/:id, admin = tier non role, creation tenant ouverte
A02 - Cryptographic Failures	MODERE	bcrypt OK, refresh hash OK, mais secrets MFA en clair, pas de chiffrement au repos
A03 - Injection	BON	Drizzle ORM parameterise, Zod validation
A04 - Insecure Design	MODERE	Pas de lockout compte, pas de verification email, mdp min 8 chars
A05 - Security Misconfiguration	BON	Headers CSP/HSTS/COOP/CORP/COEP complets
A06 - Vulnerable Components	OK	Bibliotheques modernes et maintenues
A07 - Auth Failures	MODERE	Detection brute force IP OK, mais pas de rate limit sur auth, pas de lockout compte
A08 - Data Integrity	MODERE	Zod validation OK, mais pas de CSRF, pas de rotation refresh token
A09 - Logging	BON	Audit trail fonctionnel, mais events CRUD/execution non audites
A10 - SSRF	FAIBLE	Pas de fetch URL controle par utilisateur

Tableau 8 : Analyse OWASP Top 10

9.1 JWT - Analyse Detaillee

L'implementation JWT presente plusieurs faiblesses. L'algorithme HS256 est correct, les tokens d'accès ont une expiration de 15 minutes et les refresh tokens de 7 jours avec hashage SHA-256 en base. Cependant, il manque les claims standards iss (issuer) et aud (audience), le refresh token n'est pas rotate a chaque utilisation, et la deconnexion revoke TOUS les refresh tokens d'un utilisateur au lieu de celui fourni. Le secret JWT n'a pas de longueur minimale requise et la valeur par default est "agentforge-dev-secret-change-in-production".

9.2 MFA - Implementation Reelle

La MFA est une véritable implementation TOTP, pas un stub. Elle utilise HMAC-SHA1 avec codes a 6 chiffres, fenetre de temps de 30 secondes avec tolerance de +/-1, comparaison timing-safe via `crypto.timingSafeEqual`, codes de secours a usage unique, et flux d'activation en 2 etapes (setup + verify). Cependant, les secrets MFA sont stockes en clair dans la base de donnees, ce qui les expose en cas de compromission de la DB. Les codes de secours sont egalement stockes en clair.

10. Audit 8 : Observabilite

Score: 35/100

L'observabilite est le domaine le plus deconnecte de l'application. L'infrastructure OpenTelemetry est correctement initialisee (SDK, BatchSpanProcessor, OTLP HTTP, graceful shutdown) mais aucun service de telemetrie n'est invoque depuis la logique metier. Les methodes `metricsService.recordExecution()`, `metricsService.recordCacheHit()`, `metricsService.recordLLMCallDuration()` et `tracingService.withSpan()` ne sont appelees nulle part dans le code de production. Le `requestLogger` middleware utilise `console.log()` au lieu de creer des spans ou d'enregistrer des metriques. L'`AlertManager` possede 7 regles par default bien concues, mais `evaluateMetric()` n'est jamais appele. En resume, 100% de l'infrastructure d'observabilite est du shelfware : bien installe sur l'etagere, jamais utilise.

Composant	Implementation	Integre dans le code metier
TracingService	Oui (172 lignes)	NON - zero spans crees
MetricsService	Oui (212 lignes, 8 instruments)	NON - zero metriques enregistrees
AlertManager	Oui (320 lignes, 7 regles)	NON - evaluateMetric() jamais appele
OTel SDK Init	Oui (index.ts ligne 10)	Oui - initialise correctement
Request Logger	Oui (middleware)	console.log() au lieu de spans
Dashboard	NON	Seulement des endpoints statut statiques

Tableau 9 : Statut d'integration de l'observabilite

11. Audit 9 : Performance

Score: **30/100**

Le module Performance souffre du meme probleme systemique que l'observabilite : les services sont implements mais jamais integres. Le JobQueue (530 lignes, Redis ZSET, retry, concurrence) n'est utilise par aucune route. Le DistributedCache (414 lignes, L1+L2, Pub/Sub invalidation) n'est pas utilise alors que l'ancien CacheManager plus simple est toujours en service. Le RequestDeduplicator (366 lignes) n'est applique a aucun endpoint. En consequence, les executions d'agents bloquent le worker HTTP de maniere synchrone, il n'y a pas de backpressure, pas de circuit breaker pour les appels LLM, et le CacheManager cree une nouvelle connexion Redis par instance sans pool partage.

11.1 Problemes de Conception pour la Production

Probleme	Impact	Correction
JobQueue dequeue non atomique	Double dequeue en concur.	Utiliser ZPOPMIN ou script Lua
JobQueue stats O(N)	N+1 requetes Redis	Maintenir des compteurs separes
DistributedCache vs Cluster	Echec en mode cluster	Remplacer scanStream par commande slot-aware
RequestDeduplicator busy-wait	Polling 200ms CPU-wasteful	Utiliser Redis Pub/Sub ou BLPOP
WarmPoolManager exec()	Injection de commandes	Utiliser dockerode au lieu de child_process
SandboxScheduler en memoire	Perte au redemarrage	Persister la file dans Redis
Aucun test de charge	Performance inconnue	Ajouter k6/Artillery scripts

Tableau 10 : Problemes de performance identifie

12. Audit 10 : Production Readiness

Score: **32/100**

Dimension	Score	Details
Docker (multi-stage, non-root, HC)	35/100	Multi-stage oui, pas de USER node, pas de health check API
Docker Compose	50/100	4/5 services, ports exposes, pas de limites ressources
CI/CD Pipeline	0/100	ZERO pipeline - aucun workflow GitHub Actions fonctionnel
Kubernetes/Helm	0/100	ZERO manifeste K8s, aucun Helm chart
TLS/HTTPS	10/100	Pas de certificat, Nginx en HTTP uniquement
Backup/DR	0/100	Aucun script de sauvegarde, aucun plan de reprise
Scaling horizontal	30/100	Etat en memoire bloque la mise a l'echelle
Configuration env	65/100	Zod validation, mais vars manquantes (OTEL, LOG_LEVEL)
Health checks	55/100	Endpoint unique /health, pas de liveness/readiness separe
Securite defaults	20/100	Secrets hardcodes dans compose, ports exposes

Tableau 11 : Production readiness par dimension

La plateforme n'est pas prete pour la production. Les Dockerfiles n'utilisent pas d'utilisateur non-root, Docker Compose expose tous les ports a l'hote et n'impose aucune limite de ressources, il n'y a aucun pipeline CI/CD fonctionnel (les fichiers .github/workflows existent mais ne sont pas valides), aucun manifeste Kubernetes, pas de TLS/SSL, pas de strategie de sauvegarde, et l'etat en memoire dans de nombreux services empeche la mise a l'echelle horizontale. Le CloudflareDeploymentEngine stocke son etat en Map en memoire avec un commentaire explicite : "In-memory store for deployments (in production, use DB)".

13. Failles Critiques

#	Faille	Impact	Priorite
F1	Donnees tenant non persistees (Map en memoire)	Perte totale au redemarrage	P0
F2	Aucun rate limiting sur les routes auth	Brute force, credential stuffing	P0
F3	Telemetrie sans authentification (3 endpoints)	Mutation d'alertes, fuite config	P0
F4	Fuite prefix/longueur des cles API admin	Attaque brute force ciblee	P0
F5	IDOR sur GET /projects/:id	Lecture de projets d'autres utilisateurs	P0
F6	Creation tenant ouverte a tout utilisateur	Provisionnement illimite	P1
F7	Pas de verification propriete sur /deployments/*	Modification de deploiements tiers	P1
F8	Secrets MFA stockes en clair en DB	Exposition si DB compromis	P1
F9	Admin auth = tier entreprise (pas RBAC)	Acces admin non autorise	P1
F10	Aucune isolation tenant sur routes core	Contournement des limites tenant	P1
F11	Services de performance non integres	Execution synchrone bloquante	P2
F12	Observabilite 100% shelfware	Zero visibilite en production	P2
F13	Pas de pipeline CI/CD	Aucune automatisation de deploiement	P2
F14	Dockerfiles sans utilisateur non-root	Conteneur root = escalade privilege	P2
F15	Pas de sauvegarde/DR	Perte de donnees irreversible	P2

Tableau 12 : Inventaire complet des failles critiques

14. Modules Non Fonctionnels

Un module est classe "non fonctionnel" lorsque son code existe mais n'est pas integre dans le chemin d'execution de l'application, ou lorsque ses fonctionnalites sont des stubs qui retournent des valeurs simulees sans valeur productive.

Module	Lignes de code	Raison de la non-fonctionnalite
Secret Rotation	158	Service initialise mais jamais appele au demarrage
Sandbox Execution	1 723	SandboxManager + WarmPool + Scheduler + Metrics deconnectes
Job Queue	530	Aucune route n'enfile de jobs ; execution synchrone uniquement
Distributed Cache	414	Ancien CacheManager utilise a la place
Request Deduplication	366	Aucune route n'applique la deduplication
AST Transform Service	546	ReflectionAgent fait ses propres fixes AST
Telemetry Integration	611	SDK init OK mais zero spans/metriques enregistrees
Alert Evaluation	320	7 regles definies mais evaluateMetric() jamais appele
Admin Frontend	0	Aucune page web admin n'existe
CI/CD Pipeline	0	Fichiers workflow existent mais non valides
Kubernetes	0	Aucun manifeste ni Helm chart
Backup/DR	0	Aucun script ni plan

Tableau 13 : Modules non fonctionnels (code existant sans integration)

15. Fonctionnalites Reellement Operationnelles

Les fonctionnalites suivantes ont ete verifiees par lecture du code source et tracage des appels reels. Elles sont operationnelles et integrees dans le chemin d'execution.

Fonctionnalite	Preuve Technique
MoA 3-round (Propose/Critique/Refine)	CodeGenerator.ts lignes 129-303, Promise.allSettled
9 modeles paralleles en round Propose	agents.ts ligne 108, providerCount: 9
Vote multicriteres (qualite/cout/vitesse)	CodeGenerator.ts lignes 506-602, poids 0.4/0.3/0.3
Fusion intelligente Jaccard	CodeGenerator.ts lignes 613-672, seuil 0.7
Reflection Agent avec LLM reel	ReflectionAgent.ts ligne 91, fetch Gemini 2.0 Flash
Auto-fix 3 niveaux avec escalation	SuperAgent.ts lignes 527-571
PatternLibrary 200+ patterns	PatternLibrary.ts 2 686 lignes, importe par ReflectionAgent
RL Router epsilon-greedy	RLTrainingService.ts lignes 66-75, eps 0.05 decay 0.9995
Moteur de recompense multi-facteur	RLTrainingService.ts lignes 94-118, 4 poids
Persistence RL en base PostgreSQL	RLTrainingService.ts lignes 138-156, Drizzle insert
Cache L1 memoire + L2 Redis	CacheManager.ts, namespace agentforge:cache:*
DAG execution parallele	dag.ts, tri topologique + Promise.allSettled par niveau
SSE streaming evenements	EventManager.ts, broadcastEvent vers clients SSE
MFA TOTP avec codes de secours	MFAService.ts, HMAC-SHA1, timing-safe comparison
Detection brute force IP	ThreatDetectionService.ts, 5 tentatives/15min
Headers securite complets	securityHeaders.ts, CSP/HSTS/COOP/CORP/COEP
Audit trail en base	AuditTrailService.ts, 22 types d'evenements
Cost Optimizer avec EMA	CostOptimizer.ts, RL-inspired, budget tiers
Cost Prediction Engine	CostPredictionEngine.ts, predictions par provider
User Feedback Service	UserFeedbackService.ts, integration RL

Context Management (4 services)	ContextGraph + DependencyResolver + TokenBudget + Compression
Auth JWT + bcrypt + refresh	auth.ts, HS256, 15min access, 7j refresh, SHA-256 hash

Tableau 14 : Fonctionnalités réellement opérationnelles avec preuves

16. Ameliorations Prioritaires

Phase 1 : Securite Critique (P0) - 1 semaine

Ces correctifs doivent etre deployes en priorite absolue car ils exposent la plateforme a des attaques immediates. Ajouter l'authentification aux 3 endpoints de telemetrie, supprimer la fuite de prefix de cles API, appliquer le rate limiting sur les routes d'authentification, ajouter la verification de propriete sur GET /projects/:id, et ajouter superAdminMiddleware sur POST /tenants/. Ces corrections sont simples et rapides (quelques lignes de code chacune) mais leur impact securitaire est considerable.

Phase 2 : Integration Enterprise (P1) - 2-3 semaines

Branche les services existants dans le chemin d'execution. Persister les donnees tenant en base PostgreSQL (utiliser les tables existantes dans schema.ts). Brancher le tenantMiddleware sur les routes core. Appliquer les quotas sur les executions d'agents. Initialiser le SecretRotationService au demarrage. Integrer le RBACService dans l'authentification admin. Ajouter des verifications de propriete sur les routes deployments. Chiffrer les secrets MFA avec AES-256. Rotater le refresh token a chaque utilisation.

Phase 3 : Observabilite et Performance (P2) - 2 semaines

Integrer le TracingService dans le requestLogger middleware (creer des spans pour chaque requete HTTP). Brancher le MetricsService dans LLMRouter et l'orchestrateur (enregistrer token usage, duree d'appel LLM, statut d'execution). Activer l'AlertManager avec un evaluateur periodique. Remplacer l'ancien CacheManager par le DistributedCache. Brancher le RequestDeduplicator sur les executions d'agents. Ajouter des tests de charge k6/Artillery.

Phase 4 : Production Readiness (P3) - 3-4 semaines

Corriger les Dockerfiles (USER node, health check). Ajouter des limites ressources dans Docker Compose et retirer les ports exposes. Creer un pipeline CI/CD GitHub Actions fonctionnel (lint, test, build, deploy). Ajouter TLS/SSL via Caddy ou Nginx + certbot. Implementer des sauvegardes automatiques PostgreSQL. Creer des manifestes Kubernetes et un Helm chart. Separer les

endpoints liveness/readiness. Ajouter les variables d'environnement manquantes (OTEL_EXPORTER_OTLP_ENDPOINT, LOG_LEVEL). Implementer les endpoints admin 501 et construire les pages frontend admin manquantes.

17. Niveau de Maturite

Niveau	Score Requis	Caracteristiques
Prototype	0-30	Demo fonctionnelle, pas de securite, pas de persistance
Beta	31-55	Coeur fonctionnel, securite partielle, gaps d'integration
Production	56-75	Securite complete, observabilite, CI/CD, DR
Enterprise	76-90	Multi-tenant isole, RBAC, audit, MFA, billing reel
Enterprise Elite	91-100	Auto-healing, zero-trust, compliance, scale 10K+

Tableau 15 : Echelle de maturite et scores requis

Avec un score global de **52.2/100**, AgentForge se situe au niveau **BETA**. Le coeur IA est operationnel et sophistique (MoA, Reflection, RL Router), mais l'infrastructure enterprise est en grande partie deconnectee. Les services existent dans le code mais ne sont pas integres dans le chemin d'execution. La plateforme peut demontrer ses capacites d'orchestration IA, mais ne peut pas etre deployee en production sans les corrections des Phases 1-4 decrites ci-dessus.

Positionnement sur l'echelle de maturite

	Score
Prototype (0-30)	
BETA (31-55)	52.2
Production (56-75)	
Enterprise (76-90)	
Enterprise Elite (91-100)	

Tableau 16 : Position d'AgentForge sur l'echelle de maturite

Pour atteindre le niveau PRODUCTION (56+), il faut completer les Phases 1-3 (securite critique, integration enterprise, observabilite/performance). Pour atteindre le niveau ENTERPRISE (76+), il faut en plus une isolation multi-tenant complete, un billing reel, un RBAC enforcement global, et une couverture de tests superieure a 80%. Le niveau ENTERPRISE ELITE (91+) necessite du auto-healing, du zero-trust, de la compliance certifiee et une capacite de 10 000+

utilisateurs concurrents.